📘 BOOK 19 — LOW LEVEL DESIGN (LLD)

SOLID, UML & 9 Complete Interview Case Studies (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 19 of 24 (+1 Special: Book 15A) **Versions Covered:** Language-agnostic design + Java 17/21 implementations **Prerequisites:** Book 02 (SOLID/OOP), Book 18 (all 16 Design Patterns) **Next Book:** `20_DSA_Pattern_Mastery.md`

> ⭐ **RECRUITER-PRIORITY NOTE:** LLD interviews for a 3–5 year "Java Full Stack Developer" role are where Book 02's SOLID principles and Book 18's design patterns get tested together, live, on a whiteboard. Every case study here explicitly names which patterns it uses and why — turning two already-completed books into one demonstrable interview skill.

📖 How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ఇది ఒక "concept" పుస్తకం కాదు — ఇది ఒక **practice** పుస్తకం. Chapter 1 లో SOLID/UML ని recap చేసి, ఒక repeatable LLD-interview approach నేర్పుతాము. తర్వాత 9 chapters లో, ప్రతీది ఒక classic LLD interview question ని full గా — requirements gathering నుండి, class diagram, core Java code, design pattern justification వరకు — పరిష్కరిస్తాము.

English: This is not a "concept" book — it's a **practice** book. Chapter 1 recaps SOLID/UML and teaches a repeatable LLD-interview approach. The following 9 chapters each fully solve one classic LLD interview question — from requirements gathering, through a class diagram, to core Java code and design-pattern justification.

🎯 Learning Objectives

1. Apply SOLID principles and read/draw UML class and sequence diagrams under interview time pressure.
2. Follow a repeatable approach: clarify requirements → identify entities → apply patterns → code core classes → discuss trade-offs.
3. Solve all 9 classic LLD case studies from memory, explaining every design decision.
4. Correctly justify which Book 18 pattern fits which design problem, and why alternatives are worse.

📑 Table of Contents

Ch.	Title
3	Case Study — BookMyShow (Movie Ticket Booking)
4	Case Study — Tic-Tac-Toe
5	Case Study — Elevator System
6	Case Study — ATM System
7	Case Study — Splitwise (Expense Sharing)
8	Case Study — Library Management System
9	Case Study — Food Delivery System
10	Case Study — Payment System (Wallet + Gateway)
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — LLD Fundamentals: SOLID Recap, UML, and the Interview Approach

Telugu Explanation

LLD interview అంటే "code రాయమని అడగడం" మాత్రమే కాదు — ఇది "ఒక real-world problem ని, changeable requirements తో, clean classes గా ఎలా model చేస్తారు" అని test చేయడం. Book 02's SOLID principles ఇక్కడ ప్రతి decision కి foundation. **UML class diagram** classes/relationships (`has-a` , `is-a`) చూపిస్తుంది; **sequence diagram** ఒక operation లో objects ఎలా interact అవుతాయో time-order లో చూపిస్తుంది.

Professional English Explanation

An LLD interview isn't just "write some code" — it tests "how do you model a real-world problem, with changing requirements, as clean classes?" Book 02's SOLID principles are the foundation for every decision here. A **UML class diagram** shows classes and relationships (`has-a` , `is-a`); a **sequence diagram** shows how objects interact, in time order, during one operation.

Diagram — UML Notation Quick Reference

```
CLASS DIAGRAM RELATIONSHIPS:
  Inheritance (is-a):     Car ----|>  Vehicle           (Book 02, Ch.3)
  Interface impl:         Car ----|>  Drivable (dashed)  (Book 02, Ch.5)
  Composition (owns,
    dies together):       Car ------◆ Engine             ("has-a," Engine has no life outside
Car)
  Aggregation (has,
    can outlive):         Library ------◇ Book            (Book can exist without this specific
Library)
  Association (uses):     Driver ---------> Car           (a simple reference/usage)

SEQUENCE DIAGRAM (Time flows top to bottom):
  Customer -> ParkingLot: parkVehicle(car)
  ParkingLot -> ParkingSpot: assign(car)
  ParkingSpot -> Ticket: generate()
  ParkingLot --> Customer: return ticket
```

The Repeatable LLD Interview Approach (apply this in every chapter that follows)

```
1. CLARIFY requirements (ask 3-5 scoping questions - what's in scope, what's out)
2. IDENTIFY core entities/nouns (these become classes) and behaviors/verbs (these become
methods)
3. DEFINE relationships (inheritance? composition? aggregation? - draw the class diagram)
4. APPLY SOLID (Book 02) - does one class have too many responsibilities? (SRP)
                           can new types be added without modifying existing code? (OCP)
5. IDENTIFY design patterns (Book 18) that fit naturally - don't force-fit patterns
6. CODE the core classes (interfaces first, then key implementations)
7. DISCUSS trade-offs, concurrency concerns, and how requirements could evolve
```

Internal Working

- Interviewers deliberately probe **step 4 (SOLID)** the hardest — a working solution that violates SRP (one giant class doing everything, like Book 18's `OrderProcessingSystem`) scores far lower than a slightly incomplete solution with clean separation of concerns.
- **Composition over inheritance** (Book 02, Ch.3's guidance) is the single most common LLD-interview correction — candidates who default to deep inheritance hierarchies (`ExpressParkingSpot extends ParkingSpot extends Spot`) get pushed toward composition and interfaces instead.
- Every case study in this book explicitly names its Book 18 patterns **because interviewers expect you to name them unprompted** — saying "I'm using Strategy here for pricing because it needs to vary independently of the rest of the booking flow" is what separates a senior-level answer from a junior one that merely produces working code.

Interview Answer

"My approach to any LLD problem is: clarify scope through requirements questions, identify entities and behaviors, define their relationships as a class diagram, apply SOLID to check for responsibility violations, identify which design patterns naturally fit (without forcing one in), then code the core interfaces and classes, and finally discuss trade-offs and how the design would evolve under new requirements. This is a repeatable process I apply the same way regardless of which specific system I'm asked to design."

Coding Exercise

L1: Draw a UML class diagram distinguishing composition vs aggregation for a `Car` / `Engine` / `Driver` example. **L2:** Write a short sequence diagram for "customer places an order" using this book's approach. **L3:** Identify one SRP violation in a hypothetical class and split it correctly. **L4 (Interview):** Walk through your LLD interview approach in under 90 seconds. **L5 (Senior):** Critique a composition-vs-inheritance decision in a sample design and justify the correct choice. **L6 (Mastery):** Apply this 7-step approach, from memory, to a system not covered in this book (e.g., a vending machine).

CHAPTER 2 — Case Study: Parking Lot System

Telugu Explanation

Requirements: Multiple floors, different spot sizes (Motorcycle/Compact/Large), vehicle-to-spot matching, entry/exit tickets, fee calculation. **Core entities:** `ParkingLot`, `ParkingFloor`, `ParkingSpot` (abstract, by size), `Vehicle` (by type), `Ticket`. **Patterns:** Strategy (Book 18, Ch.12) for fee calculation (hourly vs flat-rate), Factory (Ch.2) for spot assignment logic.

Professional English Explanation

Requirements: multiple floors, different spot sizes (Motorcycle/Compact/Large), vehicle-to-spot matching, entry/exit tickets, fee calculation. **Core entities:** `ParkingLot`, `ParkingFloor`, `ParkingSpot` (abstract, by size), `Vehicle` (by type), `Ticket`. **Patterns:** Strategy (Book 18, Ch.12) for fee calculation (hourly vs flat-rate), Factory (Ch.2) for spot assignment.

Diagram — Class Structure

```
ParkingLot ◆---- ParkingFloor ◆---- ParkingSpot <|-- MotorcycleSpot, CompactSpot, LargeSpot
                                          |
                                          uses (association)
                                          v
                                     Vehicle <|-- Motorcycle, Car, Truck
ParkingLot --------> Ticket (generated per parked vehicle)
FeeCalculationStrategy <|-- HourlyFeeStrategy, FlatRateFeeStrategy   (Book 18, Ch.12 -
Strategy)
```

Java Code — Core Classes

```
enum VehicleSize { MOTORCYCLE, COMPACT, LARGE }

abstract class Vehicle {
    private final String licensePlate;
    private final VehicleSize size;
    Vehicle(String licensePlate, VehicleSize size) { this.licensePlate = licensePlate;
this.size = size; }
    VehicleSize getSize() { return size; }
}
class Car extends Vehicle { Car(String plate) { super(plate, VehicleSize.COMPACT); } }
class Motorcycle extends Vehicle { Motorcycle(String plate) { super(plate,
VehicleSize.MOTORCYCLE); } }

class ParkingSpot {
    private final String id;
    private final VehicleSize size;
    private Vehicle parkedVehicle;

    ParkingSpot(String id, VehicleSize size) { this.id = id; this.size = size; }

    boolean canFit(Vehicle vehicle) { return parkedVehicle == null &&
vehicle.getSize().ordinal() <= size.ordinal(); }
    void park(Vehicle vehicle) { this.parkedVehicle = vehicle; }
    void vacate() { this.parkedVehicle = null; }
}

class ParkingFloor {
    private final List<ParkingSpot> spots;
    ParkingFloor(List<ParkingSpot> spots) { this.spots = spots; }

    Optional<ParkingSpot> findAvailableSpot(Vehicle vehicle) {                  // Book 06 -
Optional, not null
        return spots.stream().filter(s -> s.canFit(vehicle)).findFirst();          // Book 07 -
Streams
    }
}

interface FeeCalculationStrategy { BigDecimal calculate(Duration parkedDuration); }   // Book
18, Ch.12 - Strategy

class HourlyFeeStrategy implements FeeCalculationStrategy {
    public BigDecimal calculate(Duration duration) {
        long hours = Math.max(1, duration.toHours());                                 //
minimum 1 hour charged
        return BigDecimal.valueOf(hours).multiply(new BigDecimal("30"));
    }
}

class ParkingLot {
    private final List<ParkingFloor> floors;
    private final FeeCalculationStrategy feeStrategy;                                  //
injected - swappable (Ch.12)
    private final Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();        // Book
08 - thread-safe

    Ticket parkVehicle(Vehicle vehicle) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findAvailableSpot(vehicle);
```

```
            if (spot.isPresent()) {
                spot.get().park(vehicle);
                Ticket ticket = new Ticket(vehicle, spot.get(), Instant.now());
                activeTickets.put(ticket.getId(), ticket);
                return ticket;
            }
        }
        throw new IllegalStateException("Parking lot full for this vehicle size");
    }

    BigDecimal processExit(String ticketId) {
        Ticket ticket = activeTickets.remove(ticketId);
        ticket.getSpot().vacate();
        Duration parkedFor = Duration.between(ticket.getEntryTime(), Instant.now());
        return feeStrategy.calculate(parkedFor);                                    //
Strategy - swappable pricing
    }
}
```

Internal Working

- `canFit()` uses `VehicleSize.ordinal()` comparison so a motorcycle can also park in a compact/large spot (graceful downgrade), but a truck cannot fit in a compact spot — this single method encodes the whole matching rule instead of scattering it across `if-else` chains.
- `activeTickets` is a `ConcurrentHashMap` (Book 08) because a real parking lot has concurrent entries/exits across multiple gates — a plain `HashMap` here would be a genuine production race-condition bug, not just a style nitpick.
- `FeeCalculationStrategy` being injected (not hardcoded) means switching from hourly to flat-rate pricing (a very common LLD-interview follow-up question) requires zero changes to `ParkingLot` — exactly Book 18, Ch.12's Strategy benefit.

Real-World Example

Airport parking systems use precisely this floor→spot→vehicle-size matching logic, often layered with dynamic (surge) pricing strategies swapped by time-of-day — a direct real-world instance of the `FeeCalculationStrategy` abstraction.

Interview Answer

"I model `ParkingLot` as composing multiple `ParkingFloor`s, each composing `ParkingSpot`s sized by `VehicleSize`. Matching uses an ordinal comparison so smaller vehicles can use larger spots but not vice versa. Fee calculation is a Strategy (Book 18) injected into `ParkingLot`, so pricing models can change without touching parking/ticketing logic. Concurrent gate entries are handled with a thread-safe map for active tickets, since a real parking lot has genuine concurrent access."

Cross Questions

- Q: Why use `ConcurrentHashMap` instead of `HashMap` for `activeTickets`? → A: Multiple entry/exit gates operate concurrently in a real system; a plain `HashMap` isn't thread-safe and could corrupt state or throw under concurrent modification.

- Q: How does the design support adding a new vehicle size without changing `ParkingSpot.canFit()`? → A: It doesn't fully — this is a legitimate follow-up: `VehicleSize`'s ordinal-based comparison assumes a linear size hierarchy; a truly extensible version would need explicit compatibility rules rather than enum ordinals, which is worth raising proactively as a design trade-off.

Tricky Questions

- Q: What happens if two threads call `parkVehicle()` for the last available spot simultaneously? → A: As written, there's a race condition inside `findAvailableSpot` + `park()` — this needs either spot-level locking or an atomic reserve-then-park operation; flagging this unprompted is a strong senior-level signal.

Coding Exercise

L1: Implement `ParkingSpot`, `ParkingFloor`, and vehicle-size matching. **L2:** Implement `HourlyFeeStrategy` and a second `FlatRateFeeStrategy`. **L3:** Add a `Ticket` class with entry time, spot, and vehicle. **L4 (Interview):** Explain the design end-to-end in under 3 minutes. **L5 (Senior):** Fix the race condition identified in the Tricky Question above. **L6 (Mastery):** Extend the design to support reserved/VIP spots without modifying `ParkingSpot`.

CHAPTER 3 — Case Study: BookMyShow (Movie Ticket Booking)

Telugu Explanation

Requirements: Movies, theaters, shows (movie+theater+time), seat selection, concurrent booking (అదే seat ని ఇద్దరు book చేయకూడదు), payment. **Core entities:** `Movie`, `Theater`, `Show`, `Seat`, `Booking`. **Key challenge:** concurrency — ఇద్దరు users ఒకే seat ని ఒకేసారి book చేయడానికి ప్రయత్నిస్తే ఏం జరుగుతుంది.

Professional English Explanation

Requirements: movies, theaters, shows (movie+theater+time), seat selection, concurrent booking (the same seat must never be double-booked), payment. **Core entities:** `Movie`, `Theater`, `Show`, `Seat`, `Booking`. **Key challenge:** concurrency — what happens when two users try to book the same seat at the same time.

Diagram — Class Structure

```
Movie <---- Show ----> Theater
              |
              ◆ (composition)
              v
          Seat[]  (each seat has a SeatStatus: AVAILABLE / LOCKED / BOOKED)

Booking ----> Show, List<Seat>, PaymentDetails
BookingState <|-- InitiatedState, LockedState, ConfirmedState, FailedState   (Book 18, Ch.15 - State)
```

Java Code — Core Classes with Concurrency-Safe Seat Locking

```
enum SeatStatus { AVAILABLE, LOCKED, BOOKED }

class Seat {
    private final String seatId;
    private final AtomicReference<SeatStatus> status = new AtomicReference<>
(SeatStatus.AVAILABLE);  // Book 08 - atomics

    boolean tryLock() {                                                    // atomic
compare-and-set - Book 08, Ch.9
        return status.compareAndSet(SeatStatus.AVAILABLE, SeatStatus.LOCKED);    // only ONE
thread wins this race
    }
    void confirmBooking() { status.set(SeatStatus.BOOKED); }
    void releaseLock() { status.compareAndSet(SeatStatus.LOCKED, SeatStatus.AVAILABLE); }   //
for timeout/cancellation
}

class Show {
    private final Movie movie;
    private final Theater theater;
    private final Map<String, Seat> seats;                                  // seatId ->
Seat

    Optional<Booking> bookSeats(List<String> seatIds, Customer customer) {
        List<Seat> lockedSeats = new ArrayList<>();
        for (String seatId : seatIds) {
            Seat seat = seats.get(seatId);
            if (seat.tryLock()) {
                lockedSeats.add(seat);
            } else {
                lockedSeats.forEach(Seat::releaseLock);                          // rollback
ALL locks if any seat fails
                return Optional.empty();                                           // Book 04
- fail predictably, no partial booking
            }
        }
        return Optional.of(new Booking(this, lockedSeats, customer));
    }
}

class Booking {
    private final Show show;
    private final List<Seat> seats;
    private BookingState state = new InitiatedBookingState();                    // Book 18,
Ch.15 - State pattern

    void confirmPayment(PaymentResult result) {
        this.state = state.onPaymentResult(this, result);                        // delegates
to current state, exactly Ch.15
    }
}
```

Internal Working

- `AtomicReference.compareAndSet()` (Book 08, Ch.9) is the crux of this entire design — it atomically checks "is this seat AVAILABLE?" and sets it to LOCKED **in one indivisible operation**,

which is what prevents two threads from both believing they successfully locked the same seat; a naive `if (seat.getStatus() == AVAILABLE) seat.setStatus(LOCKED)` has a race window between the check and the set.

- **All-or-nothing locking**: if a multi-seat booking fails to lock even one requested seat, every already-locked seat in that request is explicitly released — this prevents a partial booking from silently holding seats hostage that the user never actually gets to pay for.
- The `BookingState` field is a direct reuse of Book 18, Ch.15's State pattern — `InitiatedBookingState → LockedState → ConfirmedState/FailedState`, with each state class enforcing its own valid transitions, exactly like Book 16, Ch.8's SAGA order-status machine.

Real-World Example

Real ticketing platforms hold a seat lock for a short window (e.g., 5–10 minutes) while the user completes payment, releasing it automatically via a scheduled timeout if payment isn't confirmed in time — the exact `releaseLock()` mechanism shown above, triggered by a timer instead of an explicit failure.

Interview Answer

"The core challenge is preventing double-booking under concurrent access. I use `AtomicReference.compareAndSet()` per seat (Book 08) so locking a seat is a single atomic operation with no race window between checking availability and claiming it. Multi-seat bookings lock all-or-nothing, rolling back any partial locks on failure. Booking progresses through explicit states (Initiated → Locked → Confirmed/Failed) using the State pattern (Book 18), which cleanly enforces valid transitions and mirrors exactly how Book 16's SAGA pattern manages distributed order state."

Cross Questions

- Q: Why is `compareAndSet` necessary instead of a plain check-then-set? → A: Check-then-set has a race window where two threads can both pass the check before either sets the new status, causing a double-booking; `compareAndSet` performs both atomically.
- Q: What happens if a 3-seat booking successfully locks 2 seats but fails on the 3rd? → A: All previously locked seats in that request are explicitly released, preventing a partial, unpayable booking from holding seats hostage.

Tricky Questions

- Q: If a user locks a seat but never completes payment, does it stay locked forever? → A: Not in a production system — a scheduled timeout releases the lock automatically after a fixed window, which this design should explicitly account for (a legitimate gap to raise proactively).

Coding Exercise

L1: Implement `Seat` with atomic lock/release/confirm operations. **L2:** Implement `Show.bookSeats()` with all-or-nothing locking and rollback. **L3:** Implement the `BookingState` hierarchy (Initiated/Locked/Confirmed/Failed). **L4 (Interview):** Explain how double-booking is prevented, end-to-end. **L5 (Senior):** Add a scheduled seat-lock timeout/release mechanism. **L6**

(Mastery): Extend the design to support dynamic seat pricing (front row vs back row) using Strategy (Book 18, Ch.12).

CHAPTER 4 — Case Study: Tic-Tac-Toe

Telugu Explanation

Requirements: NxN board, రెండు players, win/draw detection, extensible కి different board sizes/win conditions. **Core entities:** `Board`, `Player`, `Symbol`, `Game`. **Key design decision:** win-checking logic ఎలా efficient గా, మరియు board size మారినా పనిచేసేలా రాయాలి.

Professional English Explanation

Requirements: an NxN board, two players, win/draw detection, extensibility to different board sizes/win conditions. **Core entities:** `Board`, `Player`, `Symbol`, `Game`. **Key design decision:** how to write win-checking logic efficiently, and so it keeps working if the board size changes.

Java Code — Efficient Win Detection (O(1) per move, not O(N^2) per move)

```
enum Symbol { X, O, EMPTY }

class Board {
    private final int size;
    private final Symbol[][] grid;
    private final int[] rowCounts, colCounts;                        // running counts, NOT
recomputed each move
    private int diagonalCount, antiDiagonalCount;

    Board(int size) {
        this.size = size;
        this.grid = new Symbol[size][size];
        this.rowCounts = new int[size];
        this.colCounts = new int[size];
        for (Symbol[] row : grid) Arrays.fill(row, Symbol.EMPTY);
    }

    boolean placeMoveAndCheckWin(int row, int col, Symbol symbol) {
        grid[row][col] = symbol;
        int delta = (symbol == Symbol.X) ? 1 : -1;                    // encode X as +1, O as
-1 in running sums

        rowCounts[row] += delta;
        colCounts[col] += delta;
        if (row == col) diagonalCount += delta;
        if (row + col == size - 1) antiDiagonalCount += delta;

        return Math.abs(rowCounts[row]) == size || Math.abs(colCounts[col]) == size
            || Math.abs(diagonalCount) == size || Math.abs(antiDiagonalCount) == size;   //
O(1) check!
    }
}

class Game {
    private final Board board;
    private final Deque<Player> playerTurnQueue;                      // Book 05 - Deque for
turn rotation

    GameResult playTurn(int row, int col) {
        Player current = playerTurnQueue.peek();
        boolean won = board.placeMoveAndCheckWin(row, col, current.getSymbol());
        if (won) return GameResult.win(current);
        playerTurnQueue.addLast(playerTurnQueue.poll());                 // rotate turn - Book
05's Deque idiom
        return GameResult.ongoing();
    }
}
```

Internal Working

- The naive approach re-scans the entire row/column/diagonal after every move (`O(N)` per check) — the running-sum approach shown here updates 4 counters per move and checks them in **O(1)**, which is exactly the kind of optimization interviewers reward when a candidate proactively identifies and avoids the naive approach.

- Encoding `X` as `+1` and `O` as `-1` lets a single integer counter per row/column/diagonal detect a win for **either** player — if the counter's absolute value reaches `size`, that line is either all-X or all-O; this trick generalizes correctly to any board size `N`, satisfying the extensibility requirement.
- Using a `Deque` (Book 05) for turn rotation, rotating the current player to the back after their move, cleanly generalizes to more than 2 players (a legitimate extension question) without an if-else "whose turn is it" check.

Real-World Example

This exact O(1)-per-move win-detection trick (running row/column/diagonal sums) is a standard technique used in competitive programming and is precisely the kind of algorithmic insight (connecting to Book 20's DSA patterns) that turns a "just working" LLD answer into a strong one.

Interview Answer

"I model the board with running row/column/diagonal counters rather than re-scanning the board after each move, encoding one player's symbol as +1 and the other's as -1, so a single counter reaching the board size (in absolute value) detects a win for either player in O(1) instead of O(N) per move. Turn rotation uses a Deque, which naturally generalizes beyond two players. This combination keeps the design both efficient and extensible to different board sizes and player counts without structural changes."

Cross Questions

- Q: Why encode X as +1 and O as -1 instead of separate counters for each symbol? → A: It lets one shared counter per line detect a win for either player — reaching `+size` means an all-X line, `-size` means all-O — halving the state needed versus tracking each symbol separately.
- Q: How does using a `Deque` for turn order simplify extending to more players? → A: Rotating the front player to the back after each turn generalizes to any number of players without an if-else check for whose turn is next.

Tricky Questions

- Q: Does this design correctly detect a win the instant it happens, or only at the end of a full board scan? → A: Instantly — `placeMoveAndCheckWin` checks the affected row/column/diagonal counters immediately after each individual move, so the win is detected in the same call that placed the winning symbol.

Coding Exercise

L1: Implement `Board` with O(1) win detection via running counters. **L2:** Implement `Game` with Deque-based turn rotation. **L3:** Extend the design to 3 players with 3 distinct symbols. **L4 (Interview):** Explain the O(1) win-detection trick and why it beats the naive approach. **L5 (Senior):** Add draw detection (board full, no winner) without an extra full-board scan. **L6 (Mastery):** Generalize the design to a "K-in-a-row on an NxN board" variant (like Gomoku).

CHAPTER 5 — Case Study: Elevator System

Telugu Explanation

Requirements: Multiple elevators, multiple floors, up/down requests, efficient elevator dispatch (ఏ elevator ని ఎంపిక చేయాలి). **Core entities:** `Elevator`, `ElevatorController`, `Request` (floor + direction), `ElevatorState`. **Key design decision:** dispatch algorithm — ఎక్కడో ఒక request వస్తే, ఏ elevator దాన్ని handle చేయాలో ఎలా నిర్ణయిస్తారు.

Professional English Explanation

Requirements: multiple elevators, multiple floors, up/down requests, efficient elevator dispatch (which elevator to select). **Core entities:** `Elevator`, `ElevatorController`, `Request` (floor + direction), `ElevatorState`. **Key design decision:** the dispatch algorithm — given a request somewhere, how to decide which elevator handles it.

Java Code — Core Classes with Nearest-Elevator Dispatch Strategy

```
enum Direction { UP, DOWN, IDLE }
enum ElevatorState { MOVING, STOPPED, DOOR_OPEN }

class Elevator {
    private final int id;
    private int currentFloor;
    private Direction direction = Direction.IDLE;
    private final TreeSet<Integer> upRequests = new TreeSet<>();        // Book 05 - sorted
set, naturally ordered stops
    private final TreeSet<Integer> downRequests = new TreeSet<>(Comparator.reverseOrder());

    void addRequest(int floor) {
        if (floor > currentFloor) upRequests.add(floor);
        else if (floor < currentFloor) downRequests.add(floor);
    }

    void step() {                                                       // one simulation
tick
        if (direction == Direction.UP && !upRequests.isEmpty()) {
            currentFloor = upRequests.pollFirst();                          // TreeSet gives
nearest-above floor first
        } else if (!downRequests.isEmpty()) {
            currentFloor = downRequests.pollFirst();
        }
    }

    int distanceTo(int floor) { return Math.abs(currentFloor - floor); }     // used by
dispatch strategy below
}

interface DispatchStrategy { Elevator selectElevator(List<Elevator> elevators, int
requestedFloor); }  // Book 18, Ch.12

class NearestElevatorDispatchStrategy implements DispatchStrategy {
    public Elevator selectElevator(List<Elevator> elevators, int requestedFloor) {
        return elevators.stream()
            .min(Comparator.comparingInt(e -> e.distanceTo(requestedFloor)))    // Book 07 -
Comparator, Book 05 - Strategy match
            .orElseThrow();
    }
}

class ElevatorController {
    private final List<Elevator> elevators;
    private final DispatchStrategy dispatchStrategy;                          // swappable
(Ch.12)

    void handleExternalRequest(int floor, Direction direction) {
        Elevator chosen = dispatchStrategy.selectElevator(elevators, floor);
        chosen.addRequest(floor);
    }
}
```

Internal Working

- Using a `TreeSet` (Book 05) for pending requests keeps them **automatically sorted**, so the elevator always services the nearest-in-direction floor next without a manual sort on every step — `downRequests` uses a reverse-order comparator so `pollFirst()` still gives the nearest floor in that direction.
- `DispatchStrategy` is injected exactly like Ch.2's `FeeCalculationStrategy` — swapping `NearestElevatorDispatchStrategy` for a load-balancing or zone-based strategy (a very common follow-up question) requires zero changes to `ElevatorController`.
- A full production design would also model `ElevatorState` transitions (MOVING → STOPPED → DOOR_OPEN → MOVING) as a State pattern (Book 18, Ch.15), directly analogous to Ch.3's `BookingState` and Book 16, Ch.8's SAGA state machine.

Real-World Example

Modern elevator systems use "destination dispatch" — riders select their destination floor at a lobby panel before boarding, letting the controller group compatible passengers onto the same elevator — a more advanced `DispatchStrategy` implementation than the nearest-elevator approach shown here, but pluggable into the exact same interface.

Interview Answer

"Each elevator tracks pending stops in direction-ordered `TreeSet`s so the next stop is always the nearest one in the current direction of travel, without re-sorting on every step. Dispatch — deciding which elevator answers an external call — is a Strategy (Book 18), letting the algorithm (nearest-elevator, load-balanced, zone-based) be swapped without touching the controller. Elevator motion itself would be modeled as explicit states (idle/moving/door-open) using the State pattern, the same approach used for booking and order-status state machines elsewhere in this series."

Cross Questions

- Q: Why use a `TreeSet` instead of a plain `List` for pending floor requests? → A: It keeps requests automatically sorted, so the next nearest stop in the current direction is always available via `pollFirst()` / `pollLast()` without a manual sort each step.
- Q: What would need to change to support a different dispatch algorithm? → A: Only a new `DispatchStrategy` implementation — `ElevatorController` depends on the interface, not a specific algorithm, exactly Book 18, Ch.12's Strategy benefit.

Tricky Questions

- Q: Does `NearestElevatorDispatchStrategy` account for an elevator's current direction (e.g., avoid sending someone going up to an elevator already moving down past them)? → A: No, as written it only considers distance — this is a real gap worth raising proactively, since a genuinely correct dispatch strategy must also weigh direction compatibility, not just proximity.

Coding Exercise

L1: Implement `Elevator` with direction-ordered `TreeSet` request queues. **L2:** Implement `NearestElevatorDispatchStrategy` and a second, load-balanced alternative. **L3:** Model `ElevatorState` transitions using the State pattern (Book 18, Ch.15). **L4 (Interview):** Explain the dispatch strategy abstraction and why it's swappable. **L5 (Senior):** Fix the direction-compatibility gap identified in the Tricky Question. **L6 (Mastery):** Design a destination-dispatch variant that groups compatible riders onto the same elevator.

CHAPTER 6 — Case Study: ATM System

Telugu Explanation

Requirements: Card authentication, PIN validation, balance inquiry, withdrawal (cash dispensing), deposit. **Core entities:** `ATM`, `Card`, `Account`, `CashDispenser`, `ATMState`. **Key design decision:** ATM ఒక్క సమయంలో ఒక్క operation flow (insert card → enter PIN → select operation → dispense/complete) ద్వారానే వెళ్ళాలి — ఇది State pattern కి classic use case.

Professional English Explanation

Requirements: card authentication, PIN validation, balance inquiry, withdrawal (cash dispensing), deposit. **Core entities:** `ATM`, `Card`, `Account`, `CashDispenser`, `ATMState`. **Key design decision:** an ATM must move through exactly one operation flow at a time (insert card → enter PIN → select operation → dispense/complete) — a classic State pattern use case.

Java Code — ATM Modeled as an Explicit State Machine

```
interface ATMState {
    ATMState insertCard(ATM atm, Card card);
    ATMState enterPin(ATM atm, int pin);
    ATMState selectWithdrawal(ATM atm, BigDecimal amount);
}

class IdleState implements ATMState {                                    // default state
    public ATMState insertCard(ATM atm, Card card) {
        atm.setCurrentCard(card);
        return new HasCardState();                                         // valid transition
    }
    public ATMState enterPin(ATM atm, int pin) { throw new IllegalStateException("Insert card
first"); }
    public ATMState selectWithdrawal(ATM atm, BigDecimal amount) { throw new
IllegalStateException("Insert card first"); }
}

class HasCardState implements ATMState {
    public ATMState insertCard(ATM atm, Card card) { throw new IllegalStateException("Card
already inserted"); }
    public ATMState enterPin(ATM atm, int pin) {
        return atm.validatePin(pin) ? new AuthenticatedState() : new IdleState();   // wrong
PIN -> eject, back to Idle
    }
    public ATMState selectWithdrawal(ATM atm, BigDecimal amount) { throw new
IllegalStateException("Enter PIN first"); }
}

class AuthenticatedState implements ATMState {
    public ATMState insertCard(ATM atm, Card card) { throw new IllegalStateException("Already
authenticated"); }
    public ATMState enterPin(ATM atm, int pin) { throw new IllegalStateException("Already
authenticated"); }
    public ATMState selectWithdrawal(ATM atm, BigDecimal amount) {
        atm.getCashDispenser().dispense(amount);                           // Ch.9's Facade-
like coordination
        return new IdleState();                                             // transaction
complete - return to Idle
    }
}

class ATM {
    private ATMState state = new IdleState();                              // Book 18, Ch.15 -
State pattern
    private Card currentCard;
    private final CashDispenser cashDispenser;

    void insertCard(Card card) { this.state = state.insertCard(this, card); }
    void enterPin(int pin) { this.state = state.enterPin(this, pin); }
    void selectWithdrawal(BigDecimal amount) { this.state = state.selectWithdrawal(this,
amount); }
}
```

Internal Working

- Every invalid operation for the current state (e.g., calling `selectWithdrawal` while still `IdleState`) throws immediately rather than silently doing nothing — this is Book 18, Ch.15's State pattern doing exactly what it did for `Order`'s status transitions: making illegal sequences structurally impossible to reach quietly.
- A wrong PIN transitions back to `IdleState` (ejecting the card) rather than staying in `HasCardState` indefinitely — modeling this explicitly as a state transition (not a retry counter buried in an if-else) makes adding a "3 wrong attempts locks the card" rule a clean, localized change to `HasCardState.enterPin()`.
- `CashDispenser` is a separate class the `AuthenticatedState` delegates to — this keeps `ATMState` implementations focused purely on **flow/transition logic**, while the actual mechanical dispensing (and its own failure modes, like "out of cash") lives in its own class, following Book 02's SRP.

Real-World Example

Real ATMs implement exactly this state-machine structure at the firmware level — card-reader, PIN-pad, and cash-dispenser hardware are all gated behind an explicit transaction-state machine to prevent, for example, cash being dispensed before authentication completes.

Interview Answer

"I model the ATM as an explicit state machine (Book 18, Ch.15) with states like Idle, HasCard, Authenticated — each state only exposes the operations valid from it, throwing for anything else, which makes illegal sequences (like withdrawing before authenticating) structurally impossible rather than merely discouraged by convention. A failed PIN transitions back to Idle rather than looping in place, which cleanly localizes future rules like attempt-limiting. Cash dispensing itself is delegated to a separate `CashDispenser` class to keep state-transition logic and hardware-interaction logic separately responsible, per SRP."

Cross Questions

- Q: Why does `HasCardState.enterPin()` return to `IdleState` on a wrong PIN instead of staying in `HasCardState`? → A: It models the real-world behavior of ejecting the card and resetting the transaction on authentication failure, and localizes future rules (like a 3-attempt lockout) to one state's transition logic.
- Q: Why is `CashDispenser` a separate class from `AuthenticatedState`? → A: Single Responsibility (Book 02) — state-transition logic and physical cash-dispensing logic (with its own failure modes like "out of cash") are different concerns that shouldn't be mixed into one class.

Tricky Questions

- Q: Does this design prevent withdrawing more than the account balance? → A: Not as shown — `selectWithdrawal` dispatches to `CashDispenser` without a balance check; that validation belongs either in `AuthenticatedState` before dispensing or inside an `Account` class the state consults — a gap worth naming proactively.

Coding Exercise

L1: Implement `IdleState`, `HasCardState`, `AuthenticatedState` with enforced transitions. **L2:** Add a wrong-PIN attempt counter that locks the card after 3 failures. **L3:** Add balance validation before dispensing cash. **L4 (Interview):** Explain why ATM flow is a strong fit for the State pattern. **L5 (Senior):** Add a `DepositState`/operation without modifying existing state classes' method signatures. **L6 (Mastery):** Design concurrent-access safety for the same account being accessed from two ATMs simultaneously.

CHAPTER 7 — Case Study: Splitwise (Expense Sharing)

Telugu Explanation

Requirements: Groups, expenses split among members (equally/exact amounts/percentages), "who owes whom how much" balance calculation, debt simplification. **Core entities:** `User`, `Group`, `Expense`, `Split` (strategy: Equal/Exact/Percentage), `Balance`. **Key challenge:** debt simplification — minimizing the number of actual transactions needed to settle everyone's balances.

Professional English Explanation

Requirements: groups, expenses split among members (equally/exact amounts/percentages), "who owes whom how much" balance calculation, debt simplification. **Core entities:** `User`, `Group`, `Expense`, `Split` (strategy: Equal/Exact/Percentage), `Balance`. **Key challenge:** debt simplification — minimizing the number of actual transactions needed to settle everyone's balances.

Java Code — Split Strategy and Debt Simplification

```
interface SplitStrategy { Map<User, BigDecimal> computeSplits(BigDecimal totalAmount,
List<User> participants); }  // Ch.12

class EqualSplitStrategy implements SplitStrategy {
    public Map<User, BigDecimal> computeSplits(BigDecimal total, List<User> participants) {
        BigDecimal share = total.divide(BigDecimal.valueOf(participants.size()), 2,
RoundingMode.HALF_UP);
        return participants.stream().collect(Collectors.toMap(u -> u, u -> share));   // Book
07 - Collectors
    }
}

class Expense {
    private final User paidBy;
    private final BigDecimal amount;
    private final Map<User, BigDecimal> splits;                            // computed via
SplitStrategy

    Expense(User paidBy, BigDecimal amount, SplitStrategy strategy, List<User> participants) {
        this.paidBy = paidBy; this.amount = amount;
        this.splits = strategy.computeSplits(amount, participants);            // Strategy
injected at creation time
    }
}

class BalanceSheet {
    private final Map<User, Map<User, BigDecimal>> owes = new HashMap<>();     //
owes.get(A).get(B) = amount A owes B

    void recordExpense(Expense expense) {
        for (var entry : expense.getSplits().entrySet()) {
            User debtor = entry.getKey();
            if (!debtor.equals(expense.getPaidBy())) {
                adjustBalance(debtor, expense.getPaidBy(), entry.getValue());       // debtor
owes payer
            }
        }
    }

    // Debt simplification: reduce N pairwise debts to the minimum number of net transactions
    List<Transaction> simplifyDebts(List<User> users) {
        Map<User, BigDecimal> netBalance = computeNetBalancePerUser(users);         // positive
= is owed, negative = owes
        PriorityQueue<Map.Entry<User, BigDecimal>> creditors =                       // Book 05
- PriorityQueue (max-heap)
            new PriorityQueue<>((a, b) -> b.getValue().compareTo(a.getValue()));
        PriorityQueue<Map.Entry<User, BigDecimal>> debtors =
            new PriorityQueue<>(Comparator.comparing(Map.Entry::getValue));          // min-
heap (most negative first)
        netBalance.forEach((user, bal) -> {
            if (bal.compareTo(BigDecimal.ZERO) > 0) creditors.add(Map.entry(user, bal));
            else if (bal.compareTo(BigDecimal.ZERO) < 0) debtors.add(Map.entry(user, bal));
        });

        List<Transaction> settlements = new ArrayList<>();
        while (!creditors.isEmpty() && !debtors.isEmpty()) {                         // greedy
match largest creditor/debtor
            var creditor = creditors.poll();
```

```
            var debtor = debtors.poll();
            BigDecimal settledAmount = creditor.getValue().min(debtor.getValue().abs());
            settlements.add(new Transaction(debtor.getKey(), creditor.getKey(),
settledAmount));

            BigDecimal remainingCredit = creditor.getValue().subtract(settledAmount);
            BigDecimal remainingDebt = debtor.getValue().add(settledAmount);
            if (remainingCredit.compareTo(BigDecimal.ZERO) > 0)
creditors.add(Map.entry(creditor.getKey(), remainingCredit));
            if (remainingDebt.compareTo(BigDecimal.ZERO) < 0)
debtors.add(Map.entry(debtor.getKey(), remainingDebt));
        }
        return settlements;
    }
}
```

Internal Working

- `SplitStrategy` (Book 18, Ch.12) cleanly separates "how is this expense divided" from "how are debts recorded" — adding `PercentageSplitStrategy` or `ExactAmountSplitStrategy` requires zero changes to `Expense` or `BalanceSheet`.
- **Debt simplification is the real algorithmic core** of this LLD question — naively, N people with pairwise debts could require up to N-1 settlement transactions in the worst case; the greedy max-creditor/max-debtor matching shown above (using two heaps, Book 05) minimizes actual settlement transactions by always settling the largest imbalance first, converging efficiently.
- Computing **net balance per user first** (rather than trying to simplify pairwise debts directly) is the key modeling insight — it reduces a complex graph-simplification problem to a simple greedy matching problem on two sorted lists (heaps).

Real-World Example

Splitwise's actual "simplify debts" feature uses this exact class of algorithm — instead of "A owes B ₹500, B owes C ₹500, C owes A ₹300" (3 transactions), net balances reduce it to the true minimum number of transactions needed to settle everyone.

Interview Answer

"Expenses use a Strategy (Book 18) to compute how the amount splits among participants — equal, exact, or percentage — decoupling split logic from balance recording. The interesting part is debt simplification: rather than simplifying pairwise debts directly, I first compute each user's net balance (owed minus owes), then greedily match the largest creditor against the largest debtor using two heaps, settling the smaller of the two amounts each round. This minimizes the number of actual settlement transactions and converges in at most N-1 transactions for N users."

Cross Questions

- Q: Why compute net balance per user before simplifying, rather than working with pairwise debts directly? → A: It reduces the problem to greedily matching sorted creditors against sorted debtors, which is far simpler and more efficient than trying to cancel out chains of pairwise IOUs directly.

- Q: What data structure enables efficiently finding the "largest creditor" and "largest debtor" each round? → A: A max-heap for creditors and a min-heap for debtors (`PriorityQueue` , Book 05) — both give O(log N) access to the next largest imbalance.

Tricky Questions

- Q: Does `EqualSplitStrategy` 's rounding (`RoundingMode.HALF_UP`) ever cause the split amounts to not exactly sum to the total? → A: Yes — dividing an odd total among 3 people can leave a rounding remainder; a correct implementation must explicitly assign the leftover cents to one participant (commonly the payer) rather than silently losing or gaining money in aggregate.

Coding Exercise

L1: Implement `EqualSplitStrategy` and `ExactAmountSplitStrategy` . **L2:** Implement `BalanceSheet.recordExpense()` and net balance computation. **L3:** Implement the greedy debt-simplification algorithm using two heaps. **L4 (Interview):** Explain the debt simplification algorithm and its time complexity. **L5 (Senior):** Fix the rounding-remainder bug identified in the Tricky Question. **L6 (Mastery):** Extend the design to support multi-currency expenses within one group.

CHAPTER 8 — Case Study: Library Management System

Telugu Explanation

Requirements: Books (multiple copies), members, checkout/return, due dates, fines for late return, search (by title/author/ISBN). **Core entities:** `Book`, `BookCopy` (individual physical copy), `Member`, `Loan`, `FineCalculator`. **Key design decision:** `Book` (catalog metadata) vs `BookCopy` (a specific physical, loanable item) must be modeled separately.

Professional English Explanation

Requirements: books (multiple copies), members, checkout/return, due dates, fines for late return, search (by title/author/ISBN). **Core entities:** `Book`, `BookCopy` (individual physical copy), `Member`, `Loan`, `FineCalculator`. **Key design decision:** `Book` (catalog metadata) and `BookCopy` (a specific physical, loanable item) must be modeled separately.

Java Code — Core Classes

```
class Book {                                                    // catalog-level metadata
- ONE per ISBN
    private final String isbn, title, author;
}

class BookCopy {                                                // ONE per physical copy
- what actually gets loaned
    private final String copyId;
    private final Book book;
    private BookCopyStatus status = BookCopyStatus.AVAILABLE;          // AVAILABLE / LOANED /
LOST
}

class Loan {
    private final BookCopy copy;
    private final Member member;
    private final LocalDate dueDate;
    private LocalDate returnedDate;

    boolean isOverdue() { return returnedDate == null && LocalDate.now().isAfter(dueDate); }
}

interface FineCalculator { BigDecimal calculateFine(Loan loan); }        // Book 18, Ch.12 -
Strategy

class PerDayFineCalculator implements FineCalculator {
    private static final BigDecimal DAILY_FINE = new BigDecimal("5");
    public BigDecimal calculateFine(Loan loan) {
        if (!loan.isOverdue()) return BigDecimal.ZERO;
        long overdueDays = ChronoUnit.DAYS.between(loan.getDueDate(), LocalDate.now());
        return DAILY_FINE.multiply(BigDecimal.valueOf(overdueDays));
    }
}

class Library {
    private final Map<String, List<BookCopy>> catalog;                  // isbn -> all
physical copies
    private final FineCalculator fineCalculator;

    Optional<Loan> checkout(String isbn, Member member) {
        List<BookCopy> copies = catalog.get(isbn);
        return copies.stream()
            .filter(c -> c.getStatus() == BookCopyStatus.AVAILABLE)
            .findFirst()
            .map(copy -> {
                copy.setStatus(BookCopyStatus.LOANED);
                return new Loan(copy, member, LocalDate.now().plusDays(14));
            });
    }

    BigDecimal returnBook(Loan loan) {
        loan.markReturned(LocalDate.now());
        loan.getCopy().setStatus(BookCopyStatus.AVAILABLE);
        return fineCalculator.calculateFine(loan);                          // Strategy -
swappable fine policy
    }
}
```

Internal Working

- The `Book` / `BookCopy` split is the single most important modeling decision in this case study: searching the catalog operates on `Book` (one entry per ISBN, regardless of how many physical copies exist), while checkout/loan operates on `BookCopy` (each physical item has its own independent availability status) — conflating these two would make it impossible to correctly track "3 of 5 copies of this title are currently loaned out."
- `FineCalculator` as a Strategy (Book 18, Ch.12) means switching from a flat per-day fine to a tiered policy (e.g., ₹5/day for the first week, ₹10/day after) requires only a new implementation, not changes to `Library.returnBook()`.
- `checkout()` returns `Optional<Loan>` (Book 06) rather than `null` or throwing, making "no copies available" an explicit, checked outcome the caller must handle rather than a silent null-pointer risk.

Real-World Example

University library systems universally separate "title record" (catalog metadata, one per book) from "item record" (one per physical barcode/copy) for precisely this reason — it's the same `Book` / `BookCopy` split modeled here.

Interview Answer

"The key modeling decision is separating `Book` (catalog-level metadata, one per ISBN) from `BookCopy` (one per physical item, with its own loan status) — without this split, tracking partial availability across multiple copies of the same title is impossible. Fine calculation is a Strategy (Book 18), so the fine policy can change independently of checkout/return logic. `checkout()` returns an `Optional<Loan>` (Book 06) to make unavailability an explicit, handled outcome rather than a null or an exception for what's actually a normal business case."

Cross Questions

- Q: Why can't `Book` alone track availability if a title has 5 physical copies? → A: Availability is per physical item, not per title — one `Book` entry would have no way to represent "3 of 5 copies are currently loaned," which is why `BookCopy` exists as a separate entity.
- Q: Why does `checkout()` return `Optional<Loan>` instead of throwing when no copies are available? → A: Unavailability is a normal, expected outcome in this domain, not an exceptional error condition — `Optional` (Book 06) models "no result" explicitly without the overhead/semantics of an exception.

Tricky Questions

- Q: Does this design handle a member trying to check out the same title twice (holding two copies simultaneously)? → A: Not as shown — that's a real business rule (often "one copy per title per member") that would need an explicit check in `checkout()` against the member's active loans, worth raising proactively.

Coding Exercise

L1: Implement `Book`, `BookCopy`, and catalog-based checkout. **L2:** Implement `PerDayFineCalculator` and a tiered alternative. **L3:** Add the one-copy-per-title-per-member rule identified in the Tricky Question. **L4 (Interview):** Explain the `Book`/`BookCopy` split and why it's necessary. **L5 (Senior):** Add a reservation/waitlist feature for fully-loaned titles. **L6 (Mastery):** Design search functionality supporting title/author/ISBN queries efficiently across a large catalog.

CHAPTER 9 — Case Study: Food Delivery System

Telugu Explanation

Requirements: Restaurants, menus, orders, delivery agent assignment, real-time order status tracking. **Core entities:** `Restaurant`, `MenuItem`, `Order`, `DeliveryAgent`, `OrderStatus`. **Key design decision:** delivery agent assignment (nearest-available agent, Ch.5's dispatch problem again) మరియు order status ని (Placed → Preparing → OutForDelivery → Delivered) State pattern తో model చేయడం.

Professional English Explanation

Requirements: restaurants, menus, orders, delivery agent assignment, real-time order status tracking. **Core entities:** `Restaurant`, `MenuItem`, `Order`, `DeliveryAgent`, `OrderStatus`. **Key design decision:** delivery agent assignment (nearest-available agent — Ch.5's dispatch problem again) and modeling order status (Placed → Preparing → OutForDelivery → Delivered) with the State pattern.

Java Code — Order State Machine and Agent Assignment Strategy

```
interface OrderState {                                                  // Book 18, Ch.15 - same
pattern as Ch.3, Ch.6
    OrderState next(FoodOrder order);
}
class PlacedState implements OrderState { public OrderState next(FoodOrder o) { return new
PreparingState(); } }
class PreparingState implements OrderState {
    public OrderState next(FoodOrder o) {
        o.assignDeliveryAgent();                                        // triggers
assignment on this exact transition
        return new OutForDeliveryState();
    }
}
class OutForDeliveryState implements OrderState { public OrderState next(FoodOrder o) { return
new DeliveredState(); } }
class DeliveredState implements OrderState { public OrderState next(FoodOrder o) { throw new
IllegalStateException("Terminal state"); } }

interface AgentAssignmentStrategy { DeliveryAgent assign(List<DeliveryAgent> available,
Location restaurantLocation); }

class NearestAgentStrategy implements AgentAssignmentStrategy {          // structurally
identical to Ch.5's dispatch
    public DeliveryAgent assign(List<DeliveryAgent> available, Location restaurantLocation) {
        return available.stream()
            .min(Comparator.comparingDouble(a ->
a.getLocation().distanceTo(restaurantLocation)))
            .orElseThrow(() -> new NoAgentAvailableException());
    }
}

class FoodOrder {
    private OrderState state = new PlacedState();
    private final Restaurant restaurant;
    private final AgentAssignmentStrategy assignmentStrategy;
    private DeliveryAgent assignedAgent;

    void advanceStatus() { this.state = state.next(this); }              // ONE method drives
the whole lifecycle

    void assignDeliveryAgent() {
        this.assignedAgent = assignmentStrategy.assign(
            restaurant.getAvailableAgentsNearby(), restaurant.getLocation());
    }
}
```

Internal Working

- Notice `advanceStatus()` is the **only** public method needed to drive the entire order lifecycle — each `OrderState` implementation itself decides both what the next state is AND what side effects (like agent assignment) happen exactly on that transition, which is a cleaner variant of the same State pattern already used in Ch.3 (Booking) and Ch.6 (ATM).
- `AgentAssignmentStrategy` here is structurally **identical** to Ch.5's `DispatchStrategy` for elevators — recognizing that "assign nearest available resource to a request" is a recurring

shape across completely different domains (elevators, delivery agents, and — with a small stretch — Book 16's load-balanced service instances) is exactly the pattern-recognition skill senior LLD interviews reward.

- Triggering `assignDeliveryAgent()` specifically inside `PreparingState.next()` (not earlier, not later) encodes a real business rule directly into the state machine: agents shouldn't be assigned the instant an order is placed (the restaurant hasn't even started preparing it), only once preparation begins.

Real-World Example

Food delivery platforms deliberately delay courier assignment until food preparation is underway (not at order placement) specifically to minimize courier idle/waiting time at the restaurant — the exact business rule encoded by triggering assignment inside `PreparingState` here.

Interview Answer

"Order status is modeled as an explicit state machine (Book 18, Ch.15), the same pattern used for booking (Ch.3) and ATM flow (Ch.6) in this book — each state determines the next state and can trigger side effects exactly on its transition, like assigning a delivery agent only once the order enters Preparing, not at Placed. Agent assignment itself uses a Strategy (Ch.12) that's structurally identical to Chapter 5's elevator dispatch strategy — recognizing that 'assign the nearest available resource to a request' is the same abstract problem across very different domains is a key LLD skill."

Cross Questions

- Q: Why is delivery-agent assignment triggered inside `PreparingState.next()` rather than `PlacedState.next()`? → A: It encodes the real business rule that couriers shouldn't be assigned (and made to wait) before the restaurant has actually started preparing the order.
- Q: How is `AgentAssignmentStrategy` structurally similar to Chapter 5's `DispatchStrategy`? → A: Both solve "assign the nearest available resource to a location-based request" — the same abstract shape applied to delivery agents versus elevators.

Tricky Questions

- Q: What happens if `NearestAgentStrategy.assign()` finds no available agents? → A: It throws `NoAgentAvailableException` as shown — a real system needs an explicit retry/queueing strategy for this case (e.g., re-attempt assignment after a delay) rather than letting the order transition fail outright, which is worth raising proactively.

Coding Exercise

L1: Implement the full `OrderState` hierarchy with `advanceStatus()` driving transitions. **L2:** Implement `NearestAgentStrategy` and compare its structure explicitly to Chapter 5's dispatch strategy. **L3:** Add a retry mechanism for the "no agent available" case. **L4 (Interview):** Explain why agent assignment is triggered specifically in `PreparingState`. **L5 (Senior):** Add order cancellation as a valid transition from `PlacedState` and `PreparingState` only, not later states. **L6 (Mastery):**

Design real-time order tracking (customer sees live status) using the Observer pattern (Book 18, Ch.11) connected to this state machine.

CHAPTER 10 — Case Study: Payment System (Wallet + Gateway)

Telugu Explanation

Requirements: Wallet balance management, multiple payment methods (Wallet/Card/UPI), transaction history, idempotency (అదే payment రెండుసార్లు process కాకూడదు — Book 16, Ch.7 / Book 17, Ch.7's idempotency key concept ఇక్కడ నేరుగా వర్తిస్తుంది). **Core entities:** `Wallet`, `PaymentMethod` (strategy), `Transaction`, `IdempotencyKey`.

Professional English Explanation

Requirements: wallet balance management, multiple payment methods (Wallet/Card/UPI), transaction history, idempotency (the same payment must never be processed twice — Book 16, Ch.7 / Book 17, Ch.7's idempotency-key concept applies directly here). **Core entities:** `Wallet`, `PaymentMethod` (strategy), `Transaction`, `IdempotencyKey`.

Java Code — Idempotent Payment Processing with Strategy-Based Methods

```
interface PaymentMethod { PaymentResult pay(BigDecimal amount, PaymentContext context); }   //
Ch.12/Book 18, Ch.12

class WalletPaymentMethod implements PaymentMethod {
    public PaymentResult pay(BigDecimal amount, PaymentContext ctx) {
        Wallet wallet = ctx.getWallet();
        if (wallet.getBalance().compareTo(amount) < 0)
            return PaymentResult.failed("Insufficient balance");
        wallet.debit(amount);                                          // Book 08 - must be
thread-safe internally
        return PaymentResult.success();
    }
}

class Wallet {
    private final AtomicReference<BigDecimal> balance;                    // Book 08 - atomic
for concurrent debits

    void debit(BigDecimal amount) {
        balance.updateAndGet(current -> {                                   // atomic read-
modify-write, Book 08, Ch.9
            if (current.compareTo(amount) < 0) throw new InsufficientBalanceException();
            return current.subtract(amount);
        });
    }
}

class PaymentProcessor {
    private final Map<String, PaymentResult> processedIdempotencyKeys = new ConcurrentHashMap<>
();  // Book 17, Ch.7 pattern

    PaymentResult processPayment(String idempotencyKey, BigDecimal amount,
                                 PaymentMethod method, PaymentContext context) {
        PaymentResult existing = processedIdempotencyKeys.get(idempotencyKey);   // idempotency
check FIRST
        if (existing != null) return existing;                                  // safe no-op
on duplicate/retry

        PaymentResult result = method.pay(amount, context);                      // Strategy
- swappable method
        processedIdempotencyKeys.put(idempotencyKey, result);                     // record
BEFORE returning
        return result;
    }
}
```

Internal Working

- `Wallet.debit()` uses `AtomicReference.updateAndGet()` (Book 08, Ch.9) rather than a plain `if (balance >= amount) balance -= amount` — this closes the exact same check-then-act race window Chapter 3's seat locking addressed: two concurrent debits against a low balance could otherwise both pass the check before either actually subtracts, over-drafting the wallet.
- The **idempotency-key check is deliberately the very first thing `processPayment` does** — this directly reuses the pattern established in Book 16, Ch.7 (payment retries) and Book 17, Ch.7

(Kafka consumer idempotency): a client retrying a timed-out request with the same idempotency key gets the original result replayed, never a second charge.

- `PaymentMethod` as a Strategy (Book 18, Ch.12) means Wallet/Card/UPI are fully interchangeable from `PaymentProcessor`'s perspective — this is the same abstraction Book 16's `PaymentProviderFactory` (Abstract Factory, Ch.3) used at the provider-family level; here it's applied at the individual-method level within one provider.

Real-World Example

Every production payment gateway (Stripe, Razorpay) requires clients to pass an idempotency key on charge requests specifically so that a network timeout — where the client doesn't know if the charge actually succeeded — can be safely retried without any risk of double-charging, exactly the mechanism implemented here.

Interview Answer

"Payment methods are a Strategy (Book 18), making Wallet/Card/UPI interchangeable behind one `PaymentMethod` interface. Wallet balance updates use `AtomicReference.updateAndGet()` (Book 08) to close the check-then-act race window that a naive balance check-and-subtract would have under concurrent debits. Most importantly, `processPayment` checks an idempotency key before doing any work and records the result before returning — this is the exact same idempotent-processing pattern established for payment retries in Book 16 and Kafka consumers in Book 17, ensuring a retried request never causes a duplicate charge."

Cross Questions

- Q: Why does `Wallet.debit()` use `updateAndGet()` instead of a plain balance check followed by subtraction? → A: The plain version has a race window where two concurrent debits could both pass the sufficient-balance check before either actually subtracts, causing an over-draft; `updateAndGet()` performs the check-and-subtract atomically.
- Q: Why must the idempotency-key check happen before calling `method.pay()`, not after? → A: To guarantee a retried request with the same key never actually re-executes the payment logic at all — checking after would already be too late, since the duplicate charge would have happened.

Tricky Questions

- Q: If two different threads call `processPayment` with the SAME idempotency key at almost the same instant, could both still execute `method.pay()`? → A: Yes, as written — `get()` then `put()` on the `ConcurrentHashMap` isn't atomic as a pair; the correct fix is `computeIfAbsent()`, which atomically checks-and-executes-if-absent — a real concurrency bug worth catching and naming.

Coding Exercise

L1: Implement `WalletPaymentMethod` and `Wallet` with atomic debit. **L2:** Implement `PaymentProcessor` with the idempotency-key check. **L3:** Fix the race condition identified in the Tricky Question using `computeIfAbsent()`. **L4 (Interview):** Explain idempotent payment processing end-to-end. **L5 (Senior):** Add `CardPaymentMethod` and `UpiPaymentMethod`, confirming zero changes

to `PaymentProcessor`. **L6 (Mastery):** Design a full refund flow that is itself idempotent, addressing partial refunds against an original transaction.

📌 FINAL REVISION NOTES

- The repeatable LLD approach (Ch.1) applies identically across all 9 case studies: clarify → entities → relationships → SOLID check → patterns → code → trade-offs.
- Concurrency correctness is a recurring, deliberately-tested theme: seat locking (Ch.3), wallet debits (Ch.10), and parking spot assignment (Ch.2) all require atomic check-and-act, never separate check-then-act.
- The State pattern (Book 18, Ch.15) recurs across Booking (Ch.3), ATM (Ch.6), and Food Delivery (Ch.9) — recognizing this recurring shape is a strong senior-level signal.
- The Strategy pattern (Book 18, Ch.12) recurs across fee calculation (Ch.2), dispatch (Ch.5), splitting (Ch.7), fines (Ch.8), agent assignment (Ch.9), and payment methods (Ch.10) — it is, by far, the single most useful pattern in LLD interviews.
- Idempotency (Ch.10) directly reuses the exact pattern established in Book 16, Ch.7 and Book 17, Ch.7 — LLD interviews reward candidates who make this cross-book connection explicitly.
- Every case study deliberately has at least one proactively-identified gap or race condition — naming these unprompted is what distinguishes a senior-level answer from a merely-working one.

🗒️ CHEAT SHEET

Case Study	Core Challenge	Primary Pattern(s)
Parking Lot	Vehicle-to-spot matching, concurrent entry	Strategy (fee)
BookMyShow	Preventing double-booking under concurrency	State (booking), atomic CAS
Tic-Tac-Toe	O(1) win detection, extensibility	Running-sum algorithmic trick
Elevator	Dispatch algorithm selection	Strategy (dispatch)
ATM	Enforcing valid operation sequences	State (transaction flow)
Splitwise	Debt simplification algorithm	Strategy (split), greedy + heaps
Library	Catalog vs physical-copy modeling	Strategy (fine), Optional
Food Delivery	Resource dispatch + status lifecycle	State (order), Strategy (assignment)
Payment System	Idempotency + atomic balance updates	Strategy (method), atomic CAS

🎤 INTERVIEW QUESTION BANK — Low Level Design

Beginner

1. What's the difference between composition and aggregation in UML?
2. Why is SRP the most commonly tested SOLID principle in LLD interviews?
3. Walk through your repeatable LLD interview approach.

Intermediate 4. Design the class structure for a Parking Lot system. 5. How would you prevent double-booking two users on the same seat? 6. Explain the `Book` / `BookCopy` split in the Library case study and why it's necessary.

Advanced 7. Design the debt-simplification algorithm for Splitwise and state its complexity. 8. Compare the State-pattern usage across Booking, ATM, and Food Delivery — what's the same, what differs? 9. Explain how idempotency is enforced in the Payment System case study and where the concurrency bug is.

Senior/Architect 10. Given a system with N resources and M requests needing nearest-resource assignment (elevators, delivery agents, parking spots), design one reusable `DispatchStrategy` abstraction serving all three. 11. Design a fully concurrent-safe version of the ATM case study for shared-account access from multiple machines. 12. Extend the Payment System case study to support idempotent partial refunds against a single original transaction.

🔁 CROSS-QUESTION ENGINE — Sample Chains

- Q: Why does seat locking (Ch.3) need `compareAndSet` instead of check-then-set? → A: Check-then-set has a race window allowing two threads to both pass the check. → Cross: Where else in this book does the identical bug class appear? → A: Wallet debits in Ch.10, and the idempotency-key check-then-put in Ch.10's Tricky Question.
- Q: Why is Strategy the most-used pattern across these case studies? → A: Nearly every domain has one responsibility (pricing, dispatch, splitting, payment method) that must vary independently and be swapped without touching surrounding logic. → Cross: Which case study uses Strategy at TWO different levels simultaneously? → A: Payment System — `PaymentMethod` (Ch.10) is Strategy at the method level, mirroring Book 16, Ch.3's Abstract Factory at the provider-family level.

🏋️ CONSOLIDATED EXERCISES (All Levels)

- Solve all 9 case studies from a blank page, from memory, applying Chapter 1's 7-step approach out loud.
- For each case study, name every Book 18 pattern used and justify why an alternative pattern would be worse.
- Identify and fix the concurrency bug proactively named in each applicable chapter (Ch.2, Ch.3, Ch.10).
- Mock-interview: have a peer pick one case study at random and time yourself completing all 7 steps in 30 minutes.

🗓️ ONE-DAY REVISION PLAN (≈6 hours)

Time	Focus
0:00–0:45	Ch.1: SOLID/UML recap, the 7-step approach
0:45–2:00	Ch.2–3: Parking Lot, BookMyShow (concurrency focus)
2:00–3:00	Ch.4–6: Tic-Tac-Toe, Elevator, ATM
3:00–4:00	Ch.7–8: Splitwise, Library
4:00–5:00	Ch.9–10: Food Delivery, Payment System
5:00–6:00	Full interview bank + one full mock LLD interview

🗓️ ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1 — approach, plus solve Parking Lot from scratch
2	Ch.3 — BookMyShow, deep focus on concurrency correctness
3	Ch.4–5 — Tic-Tac-Toe, Elevator
4	Ch.6 — ATM, deep focus on State pattern
5	Ch.7 — Splitwise, deep focus on the debt-simplification algorithm
6	Ch.8–9 — Library, Food Delivery
7	Ch.10 — Payment System + full mock interview across all 9 case studies

✅ FINAL MASTERY CHECKLIST

- [ ] I can apply the 7-step LLD approach to any new problem, unprompted.
- [ ] I can solve all 9 case studies from a blank page, from memory.
- [ ] I can name the Book 18 pattern(s) used in each case study and justify the choice.
- [ ] I can identify and fix concurrency bugs (check-then-act races) proactively.
- [ ] I can explain the recurring State-pattern shape across Booking/ATM/Food Delivery.
- [ ] I can explain the recurring Strategy-pattern shape across 6 of the 9 case studies.
- [ ] I can connect idempotency in Ch.10 back to Book 16 and Book 17's identical pattern.
- [ ] I completed a full 30-minute mock LLD interview under time pressure.

Next: `20_DSA_Pattern_Mastery.md` — Book 20, building the algorithmic pattern recognition (like Ch.4's O(1) win-detection trick and Ch.7's greedy heap-based matching) that strengthens every LLD answer's efficiency discussion.